Reproduction Guide - Direct syscalls on Sophos Endpoint (Intercept X)

Detection gap: kernel telemetry observed, zero console alerts - 0xKuraishY - September 2026

0. What this guide demonstrates

On a Windows machine protected by **Sophos Endpoint (Intercept X)** - agent running, tamper protection ON - a normal user process can execute a **direct system call** that both user-mode defense layers are supposed to catch:

1. the inline hooks placed by `SophosED.dll` on the ntdll memory/handle stubs, and
2. the HitmanPro.Alert **SysCallX64** anti-exploit mitigation, which kills processes that execute a syscall from unbacked private memory.

After the bypass, the attached PoC calls `NtQuerySystemInformation` directly, receives the **full process list (~288 KB)** through the hidden syscall, and prints it. Then the key point of this report:

Sophos Central records no event for any of it. The kernel certainly observed the syscall - it answered it - but nothing was correlated into a visible alert. That absence of correlation is the finding, not the bypass technique itself. Direct syscalls are public knowledge since 2019; an EDR console that stays silent while one executes in front of its agent has a detection gap.

Everything below is shown live, single take, in the attached video `PoC.cleaned.mp4` (recorded 2026-09-21, desktop clock visible, ending on the empty Events page).

Validation run: reproduced again on 2026-09-28 after a Windows cumulative update (build 26200.9457) and an updated SophosED build (hook filter now at SophosED.dll + 0x158A90). All nine automatic checks passed: 4 hooks verified and attributed, 329832 bytes returned through the hidden syscall (167 processes), 0 new local Sophos events. Transcript attached: `repro_output_20260928_111602.txt`. Same result on a different build: this is not version-specific.

Expected total time: **15 to 20 minutes**, most of it waiting for the Sophos installer.

> **Researcher's note.** I did this testing by hand on my own lab VM over the second and third week of September 2026. The three modifications to the public tool are small, and each one is marked with a comment in the source (the comments are in French, my working language). Nothing here requires administrator rights.

1. Background (why the silence in the console is the finding)

User-mode hooks live in the same address space, and at the same privilege level, as the code they monitor. Anything running there can be inspected, patched or bypassed from that same level. This is a structural property, and it is why direct syscalls and unhooking have been public since 2019:

- SysWhispers (Jackson_T, 2019): generated syscall stubs to skip user-mode hooks entirely.
- Hell’s Gate (am0nsec, smelly__vx, 2020): resolves syscall numbers at runtime from ntdll.
- Halo’s Gate (Reenz0h / Sektor7, 2021) and Tartarus’ Gate (Paul Laine, 2021): retrieve SSNs even from hooked stubs, by reading neighbouring clean stubs.
- Outflank (2019) documented the standard hook shape that Sophos also uses: the first 5 bytes of the API are replaced with a JMP into the security product.
- Signal Labs (2023) analysed Sophos’ own hooking engine: direct JMP on the stub, indirect JMP into the vendor DLL (hmpalert.dll in their test), original stub relocated in anonymous memory.

Sophos’ own second layer, the HitmanPro.Alert anti-exploit, enforces a SysCall policy against syscalls executed from unbacked private memory. This report shows that minor modifications to a public open-source tool defeat both layers, and that the kernel telemetry which does observe the activity is not turned into any console event. Public references are listed at the end of this guide.

2. Requirements

Item	Detail
OS	Windows 11 Pro x64, any recent build (tested: 25H2, build 26200.9445). Windows 10 x64 expected to behave the same.
Hypervisor	Tested on Proxmox. The whole mechanism lives inside the guest OS (ntdll hooks, user-mode syscalls), so the hypervisor makes no difference; Hyper-V, VMware, VirtualBox or a physical machine behave the same.
EDR	Sophos Endpoint trial with **Intercept X** (tested: Intercept X 2024.1.2.10, Core Agent 2026.2.1.3.0)
Account	**Standard user. No admin rights needed** for any step of the PoC
Files	This package: `repro.ps1`, `activebreach.dll`, `abe_poc.exe`, `SHA256SUMS.txt`
Optional	Rust 1.82+ toolchain, only if you prefer to build from source (section 5.4)

3. Part A - Build the environment

1. **Install Windows 11 x64** in a VM with default options. Take a snapshot before installing Sophos.
2. **Download the Sophos Endpoint trial** from the official Sophos site (free, no payment) and install it with the default options. Make sure **Intercept X** is included. **Reboot** when the installer asks.
3. Open the **Sophos desktop UI**. Expected: green check mark, “Your device is protected”. Tamper protection is **ON** by default; leave it on.
4. Open **Sophos Central -> Devices -> your machine** and note the versions under Assigned Products (Intercept X, Core Agent).

All four states are visible in the first minute of the attached video.

4. Part B - Baseline: verify the hooks are in place

1. Extract `abe_repro_package.zip` to `C:\abe_repro` (any folder works; `repro.ps1`, `activebreach.dll` and `abe_poc.exe` must stay together).
2. Open **PowerShell as the normal user** (no “Run as administrator”).
3. `cd C:\abe_repro`
4. Run: `powershell -ExecutionPolicy Bypass -File .\repro.ps1`

The script prints the environment, the Sophos services, then dumps the first bytes of the 6 relevant ntdll stubs. **Expected result:**

```
NtAllocateVirtualMemory   E9 A3 F2 FE BF 00 00 00   HOOK (E9 jmp)
NtProtectVirtualMemory    E9 ...                    HOOK (E9 jmp)
NtFreeVirtualMemory       E9 ...                    HOOK (E9 jmp)
NtClose                   E9 63 F3 FE BF CC CC CC   HOOK (E9 jmp)
NtCreateThreadEx          4C 8B D1 B8 C9 00 00 00   intact (native prologue)
NtQuerySystemInformation  4C 8B D1 B8 36 00 00 00   intact (native prologue)
```

A native Microsoft stub always starts with `4C 8B D1 B8` (`mov r10, rcx ; mov eax, SSN`). A stub starting with `E9` is an inline `jmp` placed by a third party - here, Sophos. The hooking is selective: only the memory-management and handle APIs are hooked, exactly the ones an EDR watches. This matches the public Signal Labs description of Sophos’ hooking.

The script then traces one hook (`NtAllocateVirtualMemory`) to its filtering code. **Expected:**

```
stub  NtAllocateVirtualMemory : 00007FF...
  E9 jmp -> trampoline page   : 00000...   (anonymous private memory)
  FF 25 indirect jmp -> filter  : 00007FF...
  filter code is INSIDE SophosED.dll + 0x5F710
```

(The exact offset varies by agent version; what matters is that the filtering code resolves **inside** **`SophosED.dll`**.)

It also counts how many processes on the machine have `SophosED.dll` loaded, to show that these hooks are permanent surveillance injected everywhere, not a reaction to the tool.

5. Part C - Run the PoC

5.0 Package layout

Everything happens in one folder. The archive extracts to this layout (it already contains the guide, the technical report and the source; you only need the two binaries, the hash file and the run transcripts next to `repro.ps1`):

```
C:\abe_repro\                          <- run everything from this folder
    activebreach.dll                    engine DLL (prebuilt, or built in 5.2)
    abe_poc.exe                         PoC executable (prebuilt, or built in 5.2)
    repro.ps1                           the automated check script
    SHA256SUMS.txt                      SHA256 of the two binaries
    source\
        ActiveBreach-Engine-custom\
            SDK\Rust\                  engine library source (the 3 patches are here)
            poc\                        PoC source (loads activebreach.dll at runtime)
```

Important: `abe_poc.exe` loads `activebreach.dll` by name, with no path. Windows resolves it from the current directory first, so **the exe and the DLL must sit in the same folder, and you must run it from that folder**. Running the exe from anywhere else fails with "Failed to load activebreach.dll".

5.1 Option A - use the prebuilt binaries (no build needed)

This is the default path. Nothing to compile.

1. Verify the hashes (or let repro.ps1 do it, it compares automatically):

```
cd C:\abe_repro
certutil -hashfile activebreach.dll SHA256
certutil -hashfile abe_poc.exe SHA256
```

Both must match SHA256SUMS.txt.

2. Go straight to 5.3 and run.

5.2 Option B - build from source

Only if you want to compile it yourself. Prerequisites:

- Rust 1.82 or newer with the `x86_64-pc-windows-msvc` target (install from rustup.rs, choose the MSVC toolchain)
- Visual Studio Build Tools 2022 (Desktop development with C++)
- Internet access during the first build (cargo downloads the dependencies)

Commands, from a normal command prompt or PowerShell:

```
:: 1. build the engine DLL (debug build: real error codes and logs)
cd C:\abe_repro\source\ActiveBreach-Engine-custom\SDK\Rust
cargo build --features ntdll_backend

:: 2. build the PoC executable
cd C:\abe_repro\source\ActiveBreach-Engine-custom\poc
cargo build

:: 3. place both binaries at the package root (they must sit together)
copy C:\abe_repro\source\ActiveBreach-Engine-custom\SDK\Rust\target\debug\activebreach.dll C:\abe_repro\
copy C:\abe_repro\source\ActiveBreach-Engine-custom\poc\target\debug\abe_poc.exe C:\abe_repro\
```

Expected: two or three warnings (unused code, unsafe block), then `Finished` for each build. Notes:

- Use the **debug** build, always. The release build of the engine masks every internal error as `0`, which turns any failure into an infinite hang with no message.
- Do not skip step 3. The build outputs land deep inside `source\...\target\debug\`; the run needs them at `C:\abe_repro\`.
- During long test sessions Sophos may quarantine one of the two binaries on disk (observed on 2026-09-28: abe_poc.exe was removed after the AAP detection flagged it). If a binary disappears, restore it from the package zip or rebuild; this removal is itself consistent with the detection being persisted as a file thumbprint.

5.3 Run the PoC

```
cd C:\abe_repro
.\abe_poc.exe
```

Expected output (verbatim capture from my run of 2026-09-17, process list shortened here):

```
[*] Launching ActiveBreach...
[AB:MAPPER] Opening C:\Windows\System32\ntdll.dll
[AB:EXPORTS] SYSCALL_TABLE initialized @0x240c4060000 (entries=488)
[AB:EXPORTS] SYSCALL_TABLE set to READONLY
[AB:EXPORTS] SYSCALL_TABLE hash = 0x17E627DBB9C53E22
[+] activebreach_launch() returns: 0
[AB:DISPATCH] dispatcher thread started (TID=4016)
[*] Calling NtQuerySystemInformation via ABE...
[AB:STUB] stub pool ready
```

```
[AB:DISPATCH][L3] (PoC debug line in French: direct ntdll prologue call)
[+] SUCCESS! Syscall executed without detection.
ReturnLength = 288280 bytes
First buffer byte: 0x50
[135] msedge.exe
... (full process list) ...
[+] 160 processes read via direct syscall.
[*] POC finished.
```

The three lines that prove each stage:

- `SYSCALL_TABLE initialized (entries=488)` - the engine initialized **even though the 4 stubs were hooked** (this is bypass layer 1).
- `[AB:DISPATCH][L3]` - the syscall is invoked **directly inside ntdll .text**, which defeats the SysCallX64 heuristic that keys on the return address landing in unbacked private memory (this is bypass layer 2).
- `ReturnLength = 288280 bytes` plus the process list - the kernel answered. The syscall really executed.

(One line of the PoC's log output is in French, my working language; it announces the direct ntdll prologue call. That string comes from my patched source, not from Sophos.)

5.4 About the exit code (expected, read this before running)

After printing `POC finished`, the process exits with code **0xC0000005** (access violation). This is expected and documented in the report (section E.3, Rigor note): the syscall and its full 288 KB response completed **before** this post-execution reaction. Please read the output above the exit line as the result, not the exit code. Notably, even this post-execution reaction produced **no event** in Sophos Central.

6. Part D - Confirm the detection gap

1. Note the time of the PoC run (the script prints start and end times).
2. Open **Sophos Central -> Devices -> your machine -> Events**.
3. Refresh. **Expected: no event** for the run - no detection, no isolation, no “suspicious behavior”, nothing. The machine still shows healthy.
4. For completeness, `repro.ps1` also diffs the **local Sophos event logs** before and after the run (it captures record-ID baselines in CHECK 7 and counts new events in CHECK 9). Expected: 0 new events.

This is the core finding. Compare with what the system certainly knew at that moment: the kernel executed the syscall and returned 288 KB of process list. Kernel callbacks (object/process/thread) and ETW Threat Intelligence exist precisely to observe such activity. Yet on the console side, nothing was correlated into an alert. The gap is not telemetry collection. It is detection and correlation.

7. Optional cross-check (A/B test)

Uninstall Sophos Endpoint, reboot, run the same `abe_poc.exe` in the same folder:

- the 6 stubs now all show `4C 8B D1 B8` (no hooks),
- the same binary succeeds and returns the process list.

Reinstall Sophos to return to the hooked state. This confirms the bypass chain is what interacts with the hooks, not some unrelated environment quirk.

8. Impact

An attacker who already has code execution on an endpoint (phishing dropper, macro, exploited browser - the common initial-access case) can use direct syscalls to perform **memory allocation and thread operations that both user-mode layers of the EDR do not see, and that produce no event in the management console**. In this test:

- SophosED user-mode hooks: **bypassed** (full initialization under 4 hooked stubs),
- HitmanPro.Alert SysCallX64 anti-exploit: **bypassed** (syscall physically executes inside ntdll .text),
- console events during the whole sequence: **0**,
- data retrieved through the hidden syscall: the full process list, 288 KB.

The user-mode layers delayed this bypass by a few hours of adaptation work on a public open-source tool; they did not stop it. What remains is kernel telemetry correlation (Ob/Ps callbacks, ETW-TI), which today produces no alert. Suggested directions, in the attached report: stack-walking inside kernel callbacks, memory scanning for homemade syscall stubs outside ntdll, integrity monitoring of the EDR's own hooks, ETW-TI correlation into visible alerts, and HVCI/VBS as hardware-level prevention.

8.1 Adaptive Attack Protection (AAP) note

During the test, Adaptive Attack Protection was OFF. That is the state this report measures, for two reasons observed on the test machine:

- AAP can only be enabled for a limited window (72 hours maximum per machine); it then turns itself off. It is a containment mode, not a standing configuration.
- Enabling it immediately started flagging legitimate applications as "controlled applications" (PowerShell, OneDrive) and cut internet access on the machine.

Retest with AAP enabled (done on 2026-09-28 at 11:19 local, inside a manually enabled window): the PoC is detected, but look at what is actually detected. Sophos Central raised a high-severity behavioral alert:

```
name     : "Disrupt_2a (T1574.002) malicious behavior detected in C:\...\abe_poc.exe"
type     : Event::Endpoint::CoreBehavioralDetection
origin   : BEHAVIORAL
threat   : Disrupt_2a, mapped to MITRE ATT&CK T1574.002 - Hijack Execution Flow: DLL Side-Loading
file ref : the alert references the SHA256 of abe_poc.exe (thumbprint 2c80615c...)
```

T1574.002 is DLL side-loading: the engine reacted to the loader shape of the PoC (an unsigned executable loading a DLL from its own directory), not to the direct-syscall activity. The syscalls themselves produced no event in either state, AAP on or off. A payload that does not match this loader shape (embedded, reflectively loaded, injected into another process) would likely not trigger this rule.

So the accurate picture is: the standing controls (user-mode hooks, SysCallX64, console correlation) are bypassed silently; the temporary containment mode can catch this specific PoC packaging, via a side-loading classification, and only inside a 72-hour window that disrupts normal usage while enabled. Detection of the technique itself - direct syscalls - is not correlated into an alert in any tested state. (The Sophos Central events visible when AAP is toggled are administrative audit events: "Adaptive Attack Protection turned on by administrator".)

9. Troubleshooting

Symptom	Cause / fix
PoC hangs forever, 0 percent CPU	You are running a **release** build of the DLL. Rebuild in debug, or use the provided binary.
`launch()` returns a large negative number, PoC aborts	Init failed. Confirm the 4 stubs show `E9` (Part B) and that you use the patched source or the provided DLL.
All 6 stubs intact	Sophos Endpoint is not installed or not running. Complete Part A.
Small ReturnLength or NTSTATUS 0xC0000004	Output buffer too small - use the provided PoC (1 MB buffer), not an older one.
Filter address not inside any module	SophosED was not injected into that PowerShell process; run the script again.
No hooks but Sophos installed	Some builds vary in their hook set; the bypass applies to whichever memory/handle stubs are hooked.
CHECK 7 prints “(no Sophos event logs found)”	Normal on some installs; local logs are optional. The authoritative check is the console (Part D).

10. Package manifest

Everything ships as a single archive, `abe_repro_package.zip`:

File / folder	Description
`REPRODUCTION_GUIDE.pdf`	This guide
`Report_ABE_Sophos_0xKuraishY_EN.pdf`	Full 15-page technical report (hook mechanism with offsets, the three bypass phases, measurements, recommendations)
`repro.ps1`	One-click environment check, hook dump, attribution, event-log diff, PoC runner with PASS/FAIL checks
`activebreach.dll`	Patched ActiveBreach-Engine v0.2.0, debug build
`abe_poc.exe`	PoC loader: init engine, call NtQuerySystemInformation via ABE, print the retrieved process list
`SHA256SUMS.txt`	SHA256 of the two binaries, from `certutil -hashfile <file> SHA256`
`repro_output_*.txt`, `poc_output.txt`	Transcripts of the validation runs (2026-09-28)
`source/ActiveBreach-Engine-custom/`	Complete patched source for review; the 3 patches are marked in `SDK/Rust/src/internal/vm.rs`, `exports.rs`, `dispatch.rs`

Setup: extract the archive to `C:\abe_repro` and follow Part B. The video `PoC.cleaned.mp4` is attached next to the archive, not inside it.

11. Public references

- Jackson_T, “SysWhispers”, 2019 - https://github.com/jthuraisamy/SysWhispers
- am0nsec and smelly__vx, “Hell’s Gate”, 2020 - https://github.com/am0nsec/HellsGate
- Reenz0h (Sektor7), “Halo’s Gate”, 2021 - https://blog.sektor7.net/#!res/2021/halosgate.md
- Paul Laine (trickster0), “Tartarus’ Gate”, 2021 - https://github.com/trickster0/TartarusGate
- Outflank, “Combining Direct System Calls and sRDI to bypass AV/EDR”, 2019 - https://www.outflank.nl/blog/2019/06/19/red-team-tactics-combining-direct-system-calls-and-srdi-to-bypass-av-edr/
- Signal Labs, “In-Memory Disassembly for EDR/AV Unhooking” (Sophos hook analysis), April 2023 - https://signal-labs.com/analysis-of-edr-hooks-bypasses-amp-our-rust-sample/
- Palantir, “Tampering with Windows Event Tracing: Background, Offense, and Defense”, December 2018 - https://blog.palantir.com/tampering-with-windows-event-tracing-background-offense-and-defense-4be7ac62ac63
- “Lifting the veil, a look at MDE under the hood” (ETW Threat Intelligence as EDR telemetry source), Troopers 2022 - https://troopers.de/downloads/troopers22/TR22_LiftingTheVeilALookAtMDE.pdf
- Virus Bulletin 2025, “Evading in plain sight: how adversaries beat user-mode protection engines for over a decade” - https://www.virusbulletin.com/uploads/pdf/conference/vb2025/papers/Evading-in-plain-sight-how-adversaries-beat-user-mode-protection-engines-for-over-a-decade.pdf

---

0xKuraishY - September 2026. All testing was done on an isolated lab VM with snapshots. No third-party systems were involved.